phone_app/lib/phone_app_web/endpoint.ex
```
plug Plug.MethodOverride
plug Plug.Head
plug Plug.Session, @session_options
plug PhoneAppWeb.Router
```

The plug function is used to define the middleware that each request goes through. You don't need to worry about all of the default plugs that Phoenix includes, but you can always view the source or remove the plug from the endpoint if needed. There are no magic (implicit) plugs in your application—only what you see in this file.

The line plug PhoneAppWeb.Router passes a request through our application-specific routing code. Let's dive into this module.

Routing Basics

Phoenix and Rails both use a central router to dispatch a request to the correct handler. These routers use a similar scheme to each other, so they feel familiar.

The Phoenix generator created the PhoneAppWeb.Router module for us. This module serves as the central router for your application, and it comes with a few lines of code to help you get started:

phone_app/lib/phone_app_web/router.ex
```
defmodule PhoneAppWeb.Router do
  use PhoneAppWeb, :router

  pipeline :browser do
    plug :accepts, ["html"]
    plug :fetch_session
    plug :fetch_live_flash
    plug :put_root_layout, {PhoneAppWeb.Layouts, :root}
    plug :protect_from_forgery
    plug :put_secure_browser_headers
  end

  pipeline :api do
    plug :accepts, ["json"]
  end

  scope "/", PhoneAppWeb do
    pipe_through :browser

    get "/", PageController, :home
  end

end
```

The first line is a use function, but it looks a bit odd. Rather than using an external Phoenix module, it's using PhoneAppWeb. This pattern is used by